

Elasticsearch 검색 품질 향상을 위한

3 단계 의미 기반 키워드 추출 파이프라인

남궁명수

Elasticsearch 기반 검색엔진 프로젝트를 진행하면서, 단순한 키워드 매칭만으로는 사용자의 검색 의도를 충분히 반영하기 어렵다는 한계를 마주했다. 특히 조사나 어미가 결합된 한국어 문장은 형태가 다양하고, 동일한 의미를 가지더라도 표현 방식이 달라 검색 품질 저하로 이어질 수 있었다. 이러한 문제를 해결하기 위해, 프로젝트에서는 텍스트를 단계적으로 처리하며 의미와 다양성을 함께 고려하는 파이프라인 구조를 설계하였다.

파이프라인은 정제 → 의미 이해 → 다양성 기반 추출의 세 단계로 구성된다. 첫 단계에서는 Kiwi 형태소 분석기를 활용하여 조사나 불필요한 표현을 제거함으로써 텍스트를 정제한다. 예를 들어 “국가재정법에”와 같은 표현은 핵심 의미를 담은 “국가재정법”으로 변환되어 이후 단계에서의 의미 분석 정확도를 높인다.

두 번째 단계에서는 Ko-SBERT 모델을 사용하여 정제된 텍스트를 고차원 벡터 공간의 좌표로 변환한다. 이 과정에서 텍스트는 단순한 문자열이 아닌, ‘법률’, ‘재정’, ‘정부’와 같은 의미적 개념 관계를 반영한 수치적 표현으로 다루어진다. 이를 통해 서로 다른 표현을 가진 단어들 간의 의미적 유사도를 계산할 수 있는 기반이 마련된다.

마지막 단계에서는 KeyBERT의 MMR(Maximal Marginal Relevance) 기법을 적용하여 키워드를 추출한다. 이 단계의 핵심은 가장 중요한 키워드를 유지하면서도, 이미 선택된 키워드와 의미적으로 지나치게 유사한 후보는 감점하고, 상대적으로 떨어져 있는 키워드에는 가산점을 부여하는 것이다. 이를 통해 ‘국가재정법’과 같은 중심 키워드뿐만 아니라 ‘청년일자리’, ‘세액공제’와 같이 회의나 문서의 전체 맥락을 드러내는 다양한 키워드를 함께 추출할 수 있다.

본 문서는 Elasticsearch 검색엔진 파이프라인을 구성하며 사용한 세 가지 모델(Kiwi, Ko-SBERT, KeyBERT)에 대해, 깊은 수학적 분석보다는 각 모델이 왜 필요하며 어떤 역할을 수행하는지에 대한 기본적인 원리 이해를 목표로 한다. 이를 통해 의미 기반 검색 구조에 대한 직관을 정리하고, 향후 검색 품질 개선을 위한 설계 판단의 근거를 마련하고자 한다.

각 모델은 이론적 세부 사항을 깊이 있게 다루기보다는, 검색엔진 파이프라인 내에서의 기능과 흐름을 중심으로 설명하며, ① 기본개념, ② 작동 원리(필요한 경우 수학적 배경 포함), ③ 실제 적용 방식과 설명 ④ 필요시 추가 설명으로 이루어진 방식으로 정리한다

텍스트 정제 단계: Kiwi 형태소 분석기

[1. Kiwi 형태소 분석기의 기본 개념]

Kiwi는 한국어 문장을 형태소 단위로 분석하기 위해 설계된 형태소 분석기이다. 한국어는 조사, 어미, 접사가 풍부하게 결합되는 교착어적 특성을 가지며, 이로 인해 동일한 의미를 가진 문장이라도 다양한 표면 형태로 표현된다. 이러한 특성은 검색엔진에서 단순 문자열 기반 매칭을 적용할 경우 검색 품질 저하의 주요 원인이 된다.

형태소 분석이란 문장을 의미를 가지는 최소 단위로 분해하고, 각 단어에 품사 정보를 부여하는 과정이다. 예를 들어 “국가재정법에”라는 표현은 표면적으로 하나의 단어처럼 보이지만, 실제로는 명사 “국가재정법”과 조사 “에”가 결합된 형태이다. Kiwi는 이러한 결합 구조를 인식하여 핵심 의미를 담은 형태소와 문법적 기능을 수행하는 요소를 구분한다.

Kiwi의 주요 목적은 문장에서 의미를 결정하는 핵심 단어를 식별하고, 검색이나 의미 분석에 불필요한 요소(조사, 일부 어미 등)를 분리하거나 제거할 수 있도록 정보를 제공하는 데 있다. 프로젝트에서는 Kiwi를 텍스트 정제 단계에 배치함으로써, 이후 단계에서 수행되는 의미 임베딩과 키워드 추출이 보다 정확한 입력을 기반으로 이어지도록 한다.

[2. Kiwi 형태소 분석기의 작동 원리]

[2-1] 문제를 인간 언어로 다시 정의

하나의 문자열이 주어진다.

– “먹었다”

사람은 즉시 다음과 같이 해석한다.

– 먹 / 었 / 다

– 동사 / 과거 / 종결

그러나 컴퓨터에게 이 문제는 전혀 다른 형태로 주어진다. 컴퓨터의 관점에서 형태소 분석이란 다음 문제로 귀결된다.

– 주어진 문자열을 생성할 수 있는 모든 형태소 시퀀스 중 가장 그럴듯한 하나를 선택하라

[2-2] 수학적으로 문제 정의하기

관측값

$X = \text{"먹었다"}$

숨겨진 값

$Y = y_1, y_2, \dots, y_k$

각 y_i 는

- 형태소 + 품사

예:

- $y_1 = \text{먹/VV}$

- $y_2 = \text{였/EP}$

- $y_3 = \text{다/EF}$

[2-3] 우리가 원하는 것

$$\hat{Y} = \arg \max_y P(Y | X)$$

“이 문자열이 주어졌을 때, 이 형태소 시퀀스가 맞을 확률이 가장 큰 것”

[2-4] 베이즈 정리를 이용한 문제 분해

$$P(Y | X) = \frac{P(X | Y) P(Y)}{P(X)}$$

여기서 $P(X)$ 는 모든 Y 에 대해 동일하므로 생략 가능하다.

따라서

$$\hat{Y} = \arg \max_y P(X | Y) P(Y)$$

[2-5] 두 확률의 의미를 정확히 분해

[2-5-1] $P(Y)$: 형태소 시퀀스가 자연스러운가?

$$P(Y) = P(y_1) \cdot P(y_2 | y_1) \cdot P(y_3 | y_2) \dots$$

이는 형태소 단위 언어 모델에 해당한다.

숫자 예시

- $P(\text{먹}) = 0.01$
- $P(\text{엎} | \text{먹}) = 0.9$
- $P(\text{다} | \text{엎}) = 0.95$

따라서, $P(Y) = 0.01 \times 0.9 \times 0.95$

[2-5-2] $P(X|Y)$: 해당 형태소들이 실제로 문자열을 만들 수 있는가?

$$P(X|Y) = \prod_i P(\text{표면 문자열} | y_i)$$

예시

- 먹 : 확률 1
- 엎 : 확률 1
- 다 : 확률 1

따라서,

$$P(X|Y) = 1$$

(형태소 결합이 불가능한 경우에는 확률 0)

[2-6] 로그 공간에서의 계산

확률 값을 그대로 곱하면 값이 매우 작아지므로(0~1), 언더플로우 발생 확률이 커지기 때문에 실제 계산에서는 로그를 사용한다.

$$\log P(X|Y) + \log P(Y)$$

이는 다음과 같이 변형된다.

$$\sum_i \log P(y_i | y_{i-1}) + \sum_i \log P(x_i | y_i)$$

즉, 곱셈 문제를 덧셈 기반 최적화 문제로 변환한 것이다.

[2-7] 가능한 형태소 시퀀스의 폭발 문제

“먹었다”에 대해 가능한 형태소 후보는 여러 가지가 존재한다.

- 먹/VV + 었/EP + 다/EF
- 먹/NNG + 었다/NNG
- 먹/VV + 었다/EF
- 기타 다수

문장이 길어질수록 가능한 경우의 수는 기하급수적으로 증가하며, 완전 탐색은 현실적으로 불가능하다.

[2-8] 형태소 그래프(DAG) 구성

이를 해결하기 위해 문자열 위에 형태소 후보, 노드를 배치하고, 가능한 전이만을 연결한 그래프를 구성한다.

(0) → 먹/VV → (1) → 었/EP → (2) → 다/EF → (3)

이 그래프는 방향 비순환 그래프(DAG) 형태를 가진다.

[2-9] 비터비(Viterbi) 알고리즘

각 상태 s 에 대해 최적 점수는 다음과 같이 계산된다.

$$V(s) = \max[V(s') + \log P(s | s') + \log P(x | s)]$$

- $V(s')$: 이전 상태까지의 최고 점수
- $\log P(s | s')$: 이전 형태소에서 지금 형태소로 가는 자연스러움
- $\log P(x | s)$: 지금 형태소가 실제 문자열을 설명하는 정수
- 과거 최고 점수 + 이번 이동 점수 + 지금 노트 점수

단계별 예시

- 먹/VV
$$V = \log P(\text{먹} / VV)$$
- 었/EP
$$V = \log P(\text{먹} / VV) + \log P(\text{였} / EP | \text{먹} / VV)$$
- 다/EF
$$V = \log P(\text{먹} / VV) + \log P(\text{였} / EP | \text{먹} / VV) + \log P(\text{다} / EF | \text{였} / EP)$$

각 단계에서는 가장 큰 값 하나만 유지한다.

[2-9] 수학적 정당성: Markov 가정

HMM 은 다음과 같은 Markov 가정을 따른다.

$$P(y_t | y_1, \dots, y_{t-1}) = P(y_t | y_{t-1})$$

이 가정으로 인해 부분 경로가 최적이면 전체 경로에서도 최적이라는 성질이 성립하므로, 동적 계획법 적용이 가능해진다.

[2-10] Kiwi 의 핵심적인 특징

Kiwi 는 모든 판단을 확률에만 의존하지 않는다.

- 언어학적으로 불가능한 형태소 연결은 사전에 제거
- 확률 모델은 남은 후보 중 최적 선택만 수행

즉, 확률 공간 자체를 먼저 축소한 뒤 최적화를 수행하는 구조이다.

[조건부 확률, 베이즈 정리]

[1] 조건부 확률 정의

$$P(Y|X) = \frac{P(X \cap Y)}{P(X)}, P(X|Y) = \frac{P(X \cap Y)}{P(Y)}$$

- $P(Y|X)$: X 가 일어났다는 조건하에 Y 가 일어날 확률
- $P(X \cap Y)$: X 도 일어나고 동시에 Y 도 일어날 경우
- $P(X)$: X 가 일어난 모든 경우

두 식에서 $P(X \cap Y)$ 를 같게 놓으면:

$$P(X \cap Y) = P(Y|X)P(X) = P(X|Y)P(Y)$$

이를 정리하면

$$P(Y|X) = \frac{P(X|Y)P(Y)}{P(X)}$$

- $P(Y|X)$: 사후확률(관측된 데이터를 본 이후에, 어떤 가설이나 사건이 참일 확률)
- $P(X|Y)$: 가능도 (X 가 관측되었을 때, Y 가 얼마나 그럴듯한가)
- $P(Y)$: 사전 확률(데이터를 보기 이전에, 어떤 사건이나 가설이 참일 거라고 믿는 정도)

[2] $P(X)$ 를 생략하는 이유

X 는 이미 관측된 값이기에 $P(X)$ 는 모든 가설 Y 에 대해 동일한 상수이다.

비교의 관점에서 보면 X 를 봤을 때, Y_1 과 Y_2 중 누가 더 가능성이 높은지에 대해 분수로 나타내면 분모 $P(X)$ 는 자동으로 소거된다.

$$\frac{P(Y_1|X)}{P(Y_2|X)} = \frac{P(X|Y_1)P(Y_1)}{P(X|Y_2)P(Y_2)}$$

따라서

$$P(Y|X) \propto P(X|Y)P(Y)$$

- $\propto$: 비례한다
- 정확한 값이 아니라 상대적 크기

[3] 직관적인 예시

- $Y = \text{스팸, 정상}$
- $X = \text{메일이 "무료" 라는 단어 포함}$

위를 통해 알고자 하는 것은 $P(\text{스팸} | \text{"무료"})$

- $P(\text{"무료"} | \text{스팸}) = 0.8$
- $P(\text{"무료"} | \text{정상}) = 0.1$
- $P(\text{스팸}) = 0.4$
- $P(\text{정상}) = 0.6$

베이즈 정리

$$P(Y | X) \propto P(X | Y)P(Y)$$

- 스팸 점수: $0.8 \times 0.4 = 0.32$
- 정상 점수: $0.1 \times 0.6 = 0.06$
- 따라서, $\hat{Y} = \text{스팸}$

[로그]

[1] 수학적으로 로그 정의

$$\log_b x = y \Leftrightarrow b^y = x$$

- b를 몇 번 곱해야 x가 되는지

[2] 로그를 둘러싼 흔한 오해

현실에서는 우리는 거의 이런 질문을 하지 않는다.

- e를 몇 제곱해야 0.000000001이 되는가

이 질문은 수학적으로는 맞지만, 인간의 사고 방식이나 실무적 판단과는 전혀 맞지 않는다.

[3] 로그의 실제 용도

로그가 사용되는 진짜 질문은

- 이 사건은 다른 사건보다 얼마나 더 말이 되는가

즉, 로그에서 중요한 것은 차이, 합, 크기 비교이다.

로그는 절대값을 해석하기 위한 도구가 아니라 비교와 누적을 가능하게 하는 표현 방식이다.

[4] 뇌에서 로그는 '지수 계산'이 아니다

HMM, 베이지 추론, 머신러닝에서 로그를 다룰 때, 다음과 같은 인식은 잘못되었다.

잘못된 인식

- 밑이 e 니까 e 를 몇 제곱해야 하지?
- 로그 값의 실제 크기를 해석해야 한다.

올바른 인식

- 로그 값은 누적 점수
- 로그 확률은 증거의 강도
- 전체 계산은 패널티의 합

[5] 로그 확률 = 패널티 시스템

확률의 성질은 단순하다

- 확률 : 항상 $0 \sim 1$
- 로그 확률: 항상 음수

위를 다음과 같이 해석하면 직관적이다.

- 각 선택은 별점을 받는다
- 자연스러운 선택 $\rightarrow$ 작은 별점(덜 음수)
- 이상한 선택 $\rightarrow$ 큰 별점(많이 음수)
- 전체 점수 = 별점의 합

따라서, 별점이 가장 적은 경로가 최선의 선택이다. (로그 확률을 쓰는 핵심 이유)

[6] 로그의 밑(base)에 대한 올바른 인식

로그의 밑은 ‘척도(scale)’일 뿐, 의미는 없다.

수식적으로,

$$\log_b x = \frac{\ln x}{\ln b}$$

- 밑을 바꾸면 → 상수 배
- 상수 배는 → 비교 결과에 영향이 없다.

따라서, 점수의 단위만 바뀔 뿐이다.(실무에서는 관례적으로 자연로그(e)를 사용한다.

[7] 비유로 이해하기

확률의 세계(원래 표현)

- 이 문장이 자연스러울 확률: 0.0000023

로그 확률의 세계(사고에 맞는 표현)

- 이 문장은 -25 점
- 다른 문장은 - 40 점

[8] 예시

문장을 생성하는 모델이 있다고 가정한다.

어떤 문장은 단어 4 개로 이루어져있고, 각 단어의 조건부 확률은 다음과 같다.

- $w_1 = 0.1$
- $w_2 = 0.01$
- $w_3 = 0.5$
- $w_4 = 0.001$

확률 세제(원래 계산)

전체 문장 확률:

$$P = 0.1 \times 0.01 \times 0.5 \times 0.001$$

$$P = 0.0000000005 = 5 \times 10^{-10}$$

- 이 숫자를 보고 좋은 문장인지 나쁜 문장인지 판단하기는 매우 어렵다.(거의 불가능)

로그를 취할 시:

$$\log P = \log 0.1 + \log 0.01 + \log 0.5 + \log 0.001$$

각각 계산하면

- $\ln(0.1) = -2.3$
- $\ln(0.01) = -4.61$
- $\ln(0.5) = -0.69$
- $\ln(0.001) = -6.91$

로그 세계 :

$$\log P = -2.3 - 4.61 - 0.69 - 6.91 = -14.51$$

- 이 문장은 -14.5 점 짜리
- 다른 문장의 점수가 있다면 누가 덜 벌점인가만 보면 된다.

[로그의 핵심 이용]

$$\log(a \times b) = \log a + \log b$$

- 곱셈 → 덧셈
- 계산이 안정적, 비교 결과는 변화 없음

$\hat{Y} = \arg \max_y P(X|Y)P(Y)$ 에 로그를 씌우면

$$\hat{Y} = \arg \max_y (\log P(X|Y) + \log P(Y))$$

로그를 쪼개면

- $P(Y) = \prod_i P(y_i | y_{i-1}) \rightarrow \log P(Y) = \sum_i \log P(y_i | y_{i-1})$: 형태소 전이 점수들의 합(문법/구조), 말이 되는 문장인가
- $P(X|Y) = \prod_i P(x_i | y_i) \rightarrow \log P(X|Y) = \sum_i \log P(x_i | y_i)$: 각 형태소가 표면 문자열을 만든 점수의 합(관측 설명), 보이는 글자를 잘 설명하는가

[DAG(Directed Acyclic Graph)]

그래프는 점(노드)과 선(간선)으로 이루어진 구조이다.

- 노드: 상태, 위치, 개체
- 간선: 이동, 관계, 연결

[Directed(방향성)]

$A \rightarrow B$

- A 에서 B 로만 갈 수 있음
- B 에서 A 로는 못 돌아감

[Acyclic(비순환)]

사이클(순환)이 없다.

$A \rightarrow B \rightarrow C \rightarrow A$ (X)

- 해당 구조 허용하지 않음

[DAG 의 핵심 특징]

- 항상 앞으로만 진행
- 되돌아가는 경로 없음
- 시작과 끝이 명확

[HMM(Hidden Markov Model)]

겉으로는 보이지 않는 상태들이 시간 순서대로 변하면서, 그 상태에 의해 관측값이 생성된다고 가정하는 확률 모델이다.

- Hidden(숨겨진 상태)
- Markov(이전 상태만 본다)
- Model(확률 모델)

[3. 실제 적용된 코드 및 설명]

Kiwi 명사 추출 함수 (Candidates 생성)

```
def get_candidates(text):  
  
    tokens = kiwi.analyze(text)  
  
    # NNG(일반명사), NNP(고유명사), SL(외국어) 포함  
  
    return [t.form for t in tokens[0][0] if t.tag in ['NNG', 'NNP', 'SL'] and len(t.form) > 1]
```

[3-1] 코드 전체 역할

```
def get_candidates(text):
```

- 입력 문장에서 “의미 후보 단어들(명사 위주)”만 뽑아내는 함수
- 이미 형태소 분석이 끝난 결과를 “활용”하는 단계

[3-2] kiwi.analyze(text)가 반환하는 것

```
tokens = kiwi.analyze(text)
```

- $P(Y)$: 형태소 전이 점수
- $P(X | Y)$: 표면 문자열 생성 점수
- Viterbi 로 가장 점수 높은 형태소 시퀀스 Y

즉, 이 결과를 받아오는 것

```
tokens = [  
  [ # 첫 번째 문장  
    ( # 1-best 결과  
      [  
        Token(form='삼성', tag='NNP'),  
        Token(form='전자', tag='NNG'),  
        Token(form='추가', tag='NNG'),  
        Token(form='상승', tag='NNG'),  
        Token(form='하', tag='VV'),  
        Token(form='있다', tag='EP+EF')  
      ],  
      score  
    )  
  ]  
]
```

[3-3] tokens[0][0]이 의미하는 것

- [0] : 첫 번째 문장
- [0] : 가장 좋은 형태소 분석 결과(1-best)

즉, Kiwi 가 확률적으로 가장 좋다고 판단한 Y

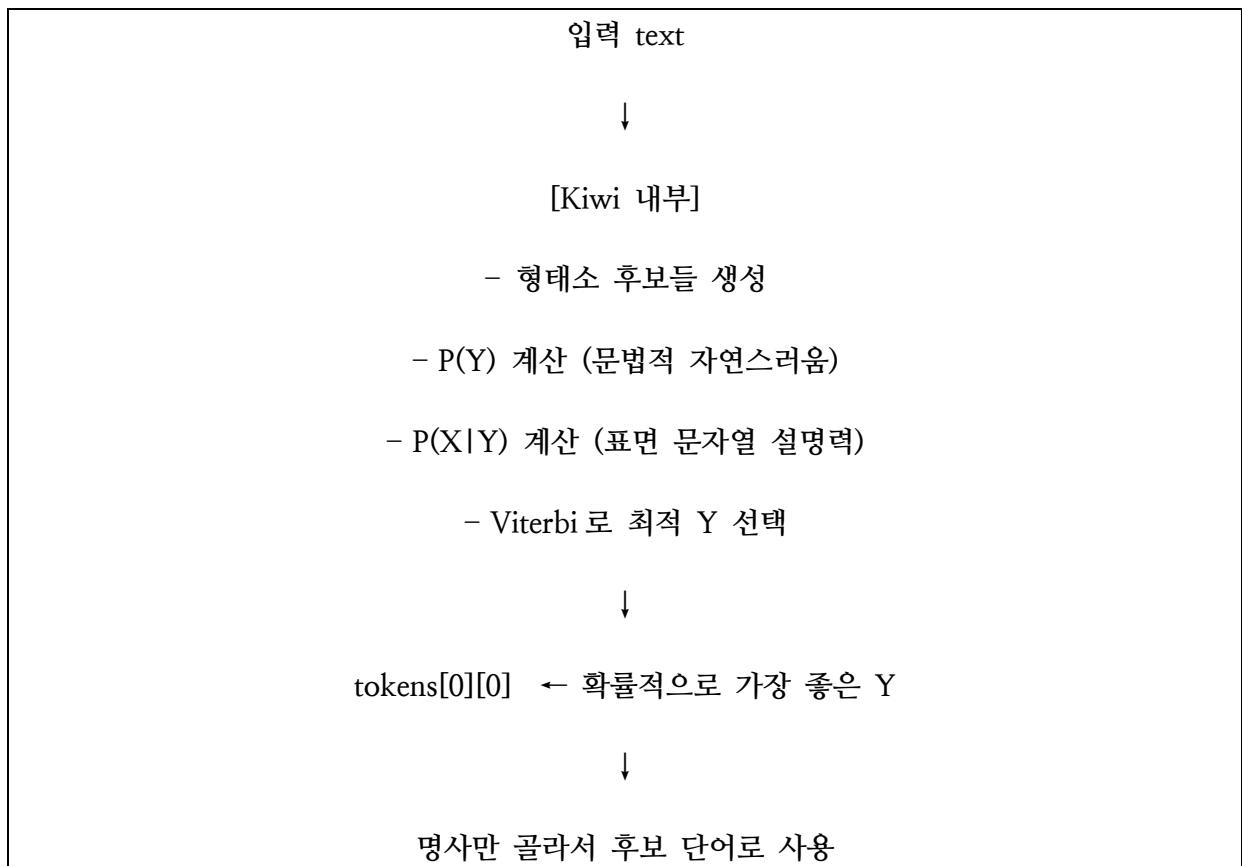
$$\hat{Y} = \arg \max_y P(X|Y)P(Y)$$

[3-4. 핵심 필터링 부분]

t.form for t in tokens[0][0] if t.tag in ['NNG', 'NNP', 'SL'] and len(t.form) > 1

- 형태소 분석 결과 중에서 일반명사/고유명사/외국어이고 길이가 2 이상인 단어만 가져오는 것

[3-5. 흐름 정리]



[4. 추가 설명]

[Kiwi 는 후보 Y (형태소 시퀀스)를 어떻게 생성하는가?]

Kiwi 는 입력 문자열이 주어지면, 먼저 사전과 규칙을 기반으로 문자열을 분할할 수 있는 모든 가능한 형태소 후보들을 생성한다. 예를 들어 문장에 명사가 포함되어 있을 경우, 해당 문자열이 일반명사(NNG), 고유명사(NNP), 외국어(SL) 중 어떤 형태소로 해석될 수 있는지를 모두 고려한다. 이렇게 생성된 형태소 후보들은 그래프 구조로 연결되며, 각 연결에는 형태소 전이 확률($P(Y)$)과 표면 문자열 생성확률 ($P(X|Y)$)이 부여된다. 이후 Kiwi 는 이 그래프에서 문법적으로 자연스럽고 관측 문자열을 가장 잘 설명하는 경로, 즉 확률 점수가 가장 높은 형태소 시퀀스 Y 를 탐색한다.

[tokens[0][1], tokens[0][2]는 무엇이 다른가]

tokens[0][1], tokens[0][2]는 각각 1-best 가 아닌 차선의 형태소 시퀀스 후보들을 의미한다. 이들은 입력 문자열을 설명할 수는 있지만, 명사 전이의 자연스러움이나 특정 명사가 해당 문자열을 생성할 가능성($P(X|Y)$)에서 1-best 후보보다 낮은 점수를 가진 해석들이다. 예를 들어 어떤 문자열이 명사(NNG)로도, 외국어(SL)로도 해석될 수 있다면, Kiwi 는 두 경우를 모두 후보 Y 로 생성하되, 확률적으로 더 그럴듯한 해석을 앞에 배치한다. 따라서 tokens[0][0]이 가장 자연스러운 명사 중심 해석이라면, 뒤의 후보들을 덜 자연스럽지만 가능한 대안 해석들이라고 볼 수 있다. 참고로 tokens[1][0]은 다음 문장이다.

[CRF (Conditional Random Field)]

입력 데이터가 주어졌을 때, 전체 출력 시퀀스의 확률을 직접 모델링하는 조건부 확률 모델이다. 형태소 분석에서는 입력 문자열 X 가 주어졌을 때, 해당 문자열에 대한 형태소 시퀀스 Y 가 얼마나 적절한지를 전체 시퀀스 단위로 평가한다. CRF 는 각 형태소를 독립적으로 판단하지 않고, 앞뒤 형태소와의 관계를 함께 고려하여 최적의 시퀀스를 결정한다는 점에서 순차 데이터 처리에 적합하다.

[HMM 과 무엇이 다른가]

HMM 은 $P(Y)$ 와 $P(X|Y)$ 를 분리하여 모델링하는 생성 모델로, 형태소 전이와 표면 문자열 생성에 대해 강한 독립 가정을 둔다. 반면 CRF 는 이러한 분리를 하지 않고, “이 입력에서 이 형태소 시퀀스가 맞는가”를 직접 판단한다. 즉, CRF 는 입력 문자열의 다양한 특징(문자 n -gram, 사전 일치 여부, 대소문자, 숫자 패턴 등)을 자유롭게 활용할 수 있어, 실제 언어 현상을 더 풍부하게 반영할 수 있다.

문맥 기반 의미 임베딩(Ko-SBERT)

[1. 문맥 기반 의미 임베딩의 기본 개념]

Ko-SBERT(Sentence-BERT)는 문장 또는 구문 단위를 의미 중심의 벡터 표현(Embedding)으로 변환하는 문맥 기반 언어 모델이다. 기존의 형태소 분석이나 빈도 기반 기법이 단어를 개별 토큰 단위로 처리하는 데 반해, Ko-SBERT는 단어들이 사용된 문맥 전체를 고려하여 의미를 수치화한다. 이를 통해 동일한 법안·정책 문서 내에서 서로 다른 표현이라 하더라도 의미적으로 유사한 개념을 하나의 의미 공간에서 인식할 수 있다.

[1-1] 문맥 기반 임베딩 의미

문맥 기반 임베딩이란, 단어 하나를 고정된 의미로 취급하지 않고 문장 내 위치와 주변 단어 관계에 따라 의미를 달리 표현하는 방식을 의미한다. 예를 들어 ‘지원’이라는 단어는 문서에 따라 재정 지원, 제도적 지원, 행정적 지원 등 서로 다른 의미를 가질 수 있는데, Ko-SBERT는 이러한 차이를 문맥을 통해 반영한다. 이로 인해 단순히 문자열이 같거나 다른지를 넘어, “어떤 맥락에서 사용되었는가”가 임베딩 결과에 직접 반영된다.

[1-2] 고차원 의미 공간으로의 매핑(Mapping)

Ko-SBERT는 입력된 문장이나 구문을 고정된 차원의 실수 벡터(일반적으로 768 차원)로 변환한다. 이는 각 텍스트를 고차원 의미 공간 상의 하나의 점으로 매핑하는 과정으로 이해할 수 있다. 이 공간에서 서로 의미적으로 유사한 문장이나 구문은 가까운 위치에 배치되고, 의미적으로 관련이 적은 경우에는 멀리 위치하게 된다. 이러한 매핑 덕분에 복합 명사가 형태소 단위로 분절되더라도, 동일한 정책·법안 맥락에 속한 표현들은 의미적으로 가까운 벡터로 표현될 수 있다.

[1-3] 형태소 기반 접근의 한계를 보완하는 역할

프로젝트에서는 Kiwi를 이용해 명사 중심의 키워드 후보를 생성하되, Ko-SBERT를 통해 이 후보들이 동일한 정책·법률 맥락에서 사용되는지 여부를 의미적으로 판단한다. 이 과정은 ‘문화관광축제’, ‘지역축제’, ‘글로벌 축제’와 같이 표면적으로는 다른 표현이지만, 문서 맥락상 밀접한 개념들을 하나의 의미 군집으로 이해하게 함으로써 복합 명사 파편화 문제를 완화하는 역할을 한다.

[2. Ko-SBERT 임베딩 공간의 구조적 정의 및 기하학적 분석]

[2-1] Ko-SBERT 의 구조적 정의: 토큰에서 문장 벡터로

Ko-SBERT(Kororean Sentence-BERT)는 사전학습된 BERT 인코더에 문장 임베딩 생성을 위한 풀링 계층(Pooling Layer)을 추가하고, 시암 네트워크(Siamese Network) 구조로 의미 유사도 학습을 수행한 모델이다. 구조적으로 Ko-SBERT 는 다음과 같은 함수 합성으로 정의된다.

$$f_{Ko-SBERT}(x) = g_{pool}(f_{BERT}(x))$$

[2-1-1] 입력 단계: 문장의 토큰화 및 임베딩

입력 문장 x 는 WordPiece 기반 토큰라이저를 통해 토큰 시퀀스로 변환되며, 각 토큰은 초기 임베딩 벡터로 사용된다.

$$x \rightarrow (t_1, t_2, \dots, t_n), \quad t_i \rightarrow e_i \in \mathbb{R}^{768}$$

따라서, 다음과 같이 표현한다.

$$x \xrightarrow{tokenizer} (t_1, t_2, \dots, t_n) \xrightarrow{embedding} (e_1, \dots, e_n), \quad e_i \in \mathbb{R}^{768}$$

각 초기 임베딩 e_i 는 임베딩, 위치 임베딩, 세그먼트 임베딩의 합으로 구성된다.

$$x_i = e_i^{token} + e_i^{position} + e_i^{segment}$$

[2-1-2] BERT 인코딩: 문맥 기반 토큰 임베딩

BERT 인코더(f_{BERT})는 Self-Attention 메커니즘을 통해 각 토큰을 문맥화된 표현(Contextualized representation)으로 변환한다.

$$(e_1, \dots, e_n) \xrightarrow{\text{Transformer Encoder}(BERT)} \{h_1, h_2, \dots, h_n\}, \quad h_i \in \mathbb{R}^{768}$$

이 단계의 H 는 토큰 수준의 정보 집합이며, 아직 문장 전체를 대표하는 단일 벡터는 아니다.

따라서, 하나의 예시로 나타내면:

- x : “문화관광축제가 열렸다”
- t_1 : 문자열 “문화”
- e_1 : 문맥 없는 초기 임베딩
- h_1 : 문맥이 반영된 “문화” 표현

[2-1-3] 풀링 계층: 정보의 응축

Ko-SBERT 는 H 위에 풀링 연산 g_{pool} 을 적용하여 고정 차원의 문장 벡터 v 를 생성한다. 정보 손실을 최소화하는 Mean Pooling 을 주로 사용한다.

$$H = g_{mean}(H) = \frac{1}{n} \sum_{i=1}^n h_i \in \mathbb{R}^{768}$$

- “문화관광축제가 열렸다”라는 문장이 들어오면, Ko-SBERT 는 각 토큰의 의미 벡터를 생성한 뒤, 이 벡터들을 평균 내어 문장 전체를 대표하는 하나의 768 차원 벡터로 풀링한다.

$$v = \frac{1}{3}(h_1 + h_2 + h_3)$$

$$v[1] = (h_1[1] + h_2[1] + h_3[1]) / 3$$

$$v[2] = (h_1[2] + h_2[2] + h_3[2]) / 3$$

- ...

$$v[768] = (h_1[768] + h_2[768] + h_3[768]) / 3$$

$$v \in \mathbb{R}^{768}$$

- “문화관광축제 + 열림(사건)”이 합쳐진 문장 전체의 의미 요약본

- “문화” 느낌

- “관광” 느낌

- “축제”라는 이벤트성

- “열렸다”라는 발생 사건

[2-1-4] Siamese Network 를 통한 의미 공간 학습

학습 단계에서는 두 개의 동일한 파라미터(θ)를 공유하는 인코더를 배치하는 시암 네트워크 구조를 채택한다.

$$v_A = f_{\theta}(x_A), \quad v_B = f_{\theta}(x_B)$$

유사한 의미를 가진 문장 쌍의 벡터가 임베딩 공간상에서 가까워지도록 거리를 최적화함으로써, 단순한 문맥 표현을 넘어선 의미적 좌표를 학습하게 된다.

[2-2] 임베딩 공간의 기하학적 실체

[2-2-1] 768 차원: 의미 표현의 자유도

768 차원은 BERT-base 의 Hidden Size 를 계승한 것으로, 단순한 숫자가 아닌 문장이 가질 수 있는 독립적인 의미 인자(Latent Factors)의 수를 의미한다. 이는 주제, 감성, 문체 등 수백 개의 독립 축이 선형 결합된 고차원 표현체이다.

[2-2-2] 단위 초구(Unit Hypersphere)로의 사상

의미 유사도 판별에 코사인 유사도(Cosine Similarity)를 사용하는 순간, 모든 문장 벡터 v 는 기하학적으로 767 차원 단위 초구($\mathbb{S}^{767}$) 위의 점으로 해석된다.

$$\hat{v} = \frac{v}{\|v\|} \rightarrow \hat{v} \in \mathbb{S}^{767}$$

고차원 초구의 특성상, 임의의 두 점은 기본적으로 직교($\cos \approx 0$)하며, 두 벡터가 가깝다는 것은 모델이 학습을 통해 도출한 매우 강력한 의미적 연관성이 있음을 수학적으로 입증한다.

- $\mathbb{S}$: 집합 또는 구면

[2-3] 실제 공간의 한계: 비등방성(Anisotropy)

이론적 정의와 달리, 실제 학습된 Ko-SBERT 공간은 모든 방향으로 균등하게 뻗어있지 않다.

- 원뿔형 분포(Cone-shaped): 대부분의 문장 벡터가 특정 방향의 좁은 구역에 밀집되는 현상이 관측된다.
- 저차원 매니폴드: 768 차원 전체를 사용하지 않고, 실제로는 훨씬 낮은 차원의 부분 공간에 데이터가 몰려 있다.
- 결과: 이로 인해 유사도 상위 키워드들이 의미적으로 중복되는 문제가 발생한다.

[2-4] 수학적 해결책: MMR(Maximal Marginal Relevance)

임베딩 공간의 비등방성으로 인한 정보 중복을 해결하기 위해, MMR 알고리즘을 통한 공간적 최적화를 수행한다.

[2-4-1] MMR 의 기하학적 정의

MMR 은 단순한 유사도 극대화가 아니라, 공간상의 ‘척력(Repulsive Force)’을 수식화한 것이다.

$$MMR = \arg \max_{D_i \in K \setminus S} [\lambda \cdot \text{sim}(D_i, Doc) - (1 - \lambda) \cdot \max_{D_j \in S} \text{sim}(D_i, D_j)]$$

- D_i : 후보 문서/문장
- S : 이미 선택된 문서/문장 집합
- K : 후보 문서 전체
- Doc : 기준 문서
- $\text{sim}(,)$: 유사도(코사인 유사도 등)
- $\lambda \in [0, 1]$: 중요도/다양성 조절 파라미터

첫 번째 부분: $\lambda \cdot \text{sim}(D_i, Doc)$

- 의미: 문서와 후보 문장의 유사도
- $\lambda = 1$ 이면 단순히 유사도 최대로 선택
- 벡터 공간에서는 $\text{sim}(D_i, Doc) = \cos(\theta_i) \rightarrow Doc$ 벡터와 후보 벡터 사이의 각도 기반 점수

즉, “이 문장이 문서 전체 의미를 잘 대표하는가”를 측정

두 번째 부분: $(1 - \lambda) \cdot \max_{D_i \in S} \text{sim}(D_i, D_j)$

- 의미: 이미 선택된 문장들과 후보 문장 간의 최대 유사도
- 최대값을 취하는 이유:
 - 한 문장이 이미 선택된 문장과 너무 비슷하면 중복 제거
- 벡터 공간에서는
 - D_j 와 D_i 벡터 사이의 각도를 계산
 - 각도가 작으면 유사도가 높고, 값이 커질수록 별점(repulsive force)

즉 “이 문장은 이미 선택된 문장들과 얼마나 겹치는가” → MMR 에서 ‘밀어내는 힘’ 역할

λ vs $(1 - \lambda)$ 의 역할

- $\lambda \uparrow$: 문서 유사도 우선, 다양성 감소
- $\lambda \downarrow$: 다양성 우선, 문서 대표성 감소

벡터 공간 비유

- 후보 벡터 = 구면 위 점
- 선택된 벡터들과 방향이 겹치면 별점, 겹치지 않으면 점수 유지
- λ 조절 = 끌어당기는 힘 vs 밀어내는 힘의 균형

최종 argmx

- 의미: MMR 기준으로 선택될 후보 벡터
- 즉, 후보 중에서 문서 대표성 높고, 이미 선택된 문장과 덜 겹치는 것을 고르는 연산
 - 문서 벡터에 가까운 후보를 뽑되
 - 이미 뽑힌 벡터들에서 멀리 떨어진 벡터를 선택
- 제 1 항(유사도): 문서 벡터와 후보 키워드 벡터 사이의 초구 상 거리를 좁힘
- 제 2 항(다양성): 이미 선택된 벡터들로부터 일정 거리 이상 떨어지도록 강제

[2-4-2] 예시: λ 기반 후보 키워드 선택

{1} 상황 설정:

문서 전체 의미 벡터: Doc

후보 키워드 벡터 $D_i \in K$ 및 문서 유사도($\text{sim}(D_i, \text{Doc})$):

후보 D_i	$\text{sim}(D_i, \text{Doc})$
“문화관광축제”	0.9
“지역축제”	0.85
“글로벌축제”	0.8
“재정”	0.7

- MMR 파라미터: $\lambda = 0.7 \rightarrow$ 문서 중요도 70%, 다양성 30%

- 초기: 이미 선택된 키워드 집합 $S = \emptyset$

{2} 1 번째 선택 ($S = \emptyset$)

$$\text{MMR}(D_i) = \lambda \cdot \text{sim}(D_i, \text{Doc}) - (1 - \lambda) \cdot \max_{D_j \in S} \text{sim}(D_i, D_j)$$

- S 가 비었으므로 별점 0

후보	$\lambda \cdot \text{sim}(D_i, \text{Doc}) - \text{별점}$	계산	MMR 점수
문화관광축제	$0.7 * 0.9 - 0$	0.63	0.63
지역축제	$0.7 * 0.85 - 0$	0.595	0.595
글로벌축제	$0.7 * 0.8 - 0$	0.56	0.56
재정	$0.7 * 0.7 - 0$	0.49	0.49

선택: “문화관광축제”

- $S = \{\text{“문화관광축제”}\}$

{3} 2 번째 선택 ($S = \{\text{"문화관광축제"}\}$)

이제 별점 계산:

$$(1 - \lambda) \cdot \max_{D_j \in S} \text{sim}(D_i, D_j)$$

후보 D_i	$\text{sim}(D_i, \text{Doc})$	$\text{sim}(D_i, S)$	$\lambda \cdot \text{sim}(D_i, \text{Doc})$	별점	MMR 점수
지역축제	0.85	0.9	0.595	$0.3 * 0.9 = 0.27$	$0.595 - 0.27 = 0.325$
글로벌축제	0.8	0.75	0.56	$0.3 * 0.75 = 0.225$	$0.56 - 0.225 = 0.335$
재정	0.7	0.1	0.49	$0.3 * 0.1 = 0.03$	$0.49 - 0.03 = 0.46$

선택: “재정”(다양성 때문에 글로벌축제보다 점수가 높음)

- $S = \{\text{"문화관광축제"}, \text{"재정"}\}$

{4} 3 번째 선택($S = \{\text{"문화관광축제"}, \text{"재정"}\}$)

- 별점 = 후보와 후보 S 안의 최대 유사도

후보 D_i	$\text{sim}(D_i, \text{Doc})$	$\text{sim}(D_i, S) = \max_{D_j \in S} \text{sim}(D_i, D_j)$	$\lambda \cdot \text{sim}(D_i, \text{Doc})$	별점	MMR 점수
지역축제	0.85	$\max(0.9, 0.2) = 0.2$	0.595	$0.3 * 0.9 = 0.27$	$0.595 - 0.27 = 0.325$
글로벌축제	0.8	$\max(0.75, 0.1) = 0.75$	0.56	$0.3 * 0.75 = 0.225$	$0.56 - 0.225 = 0.335$

선택: “글로벌축제”

- $S = \{\text{"문화관광축제"}, \text{"재정"}, \text{"글로벌축제"}\}$

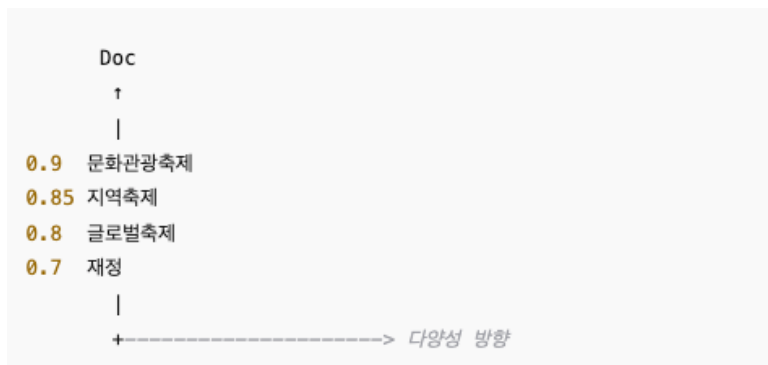
이제 후보 중 가장 대표적이고, 동시에 다양성도 확보됨

{5} λ 조절 효과

- $\lambda = 1 \rightarrow$ 다양성 무시, 문서 유사도 기준 선택 순서: “문화관광축제 $\rightarrow$ 지역축제 $\rightarrow$ 글로벌축제 $\rightarrow$ 재경”
- $\lambda = 0 \rightarrow$ 문서 유사도 무시, 완전히 다양성 기준 선택: 서로 가장 멀리 떨어진 후보부터 선택

즉, λ 는 끌어당기는 힘(문서 중심) vs 밀어내는 힘(다양성)의 균형을 조절하는 파라미터이다.

{6} 직관적 벡터 공간 비유



MMR(Maximal Marginal Relevance)은 후보 키워드 중 문서와 의미적으로 유사하면서, 이미 선택된 키워드와는 중복되지 않도록 점수를 계산하여 순차적으로 선택하는 기법이다. λ 파라미터를 통해 문서 중요도와 다양성 비율을 조절할 수 있으며, 이를 통해 의미 공간에서 문서 전체를 대표하는 다양한 키워드를 확보할 수 있다.

[2-4-3] 결론: 기저 벡터(Basis Vector)로서의 키워드

최종적으로 추출된 키워드 집합은 “문서의 의미 공간을 최소한의 개수로 가장 잘 설명하는 기저 벡터들의 집합”으로 정의될 수 있다. 이는 Ko-SBERT가 생성한 고차원 의미 공간을 가장 효율적으로 샘플링하는 과정이다.

[BERT 인코더]

BERT 인코더는 Transformer Encoder(변환기 인코더)를 다층으로 적층하여, 입력 토큰 시퀀스를 문맥 정보를 반영한 고차원 벡터 시퀀스로 변환하는 함수이다.

수식으로 쓰면:

$$f_{BERT} : (t_1, t_2, \dots, t_n) \rightarrow (h_1, \dots, h_n)$$

[Encoder]

입력을 받아 의미적으로 “압축된 표현”으로 변환하는 모듈

즉, BERT 인코더는

- 문장 생성 안함
- 다음 단어 예측 안함
- 입력 문장을 “이해”하는 역할

따라서, BERT 는 이해 모델이다.

[BERT 인코더의 입력과 출력]

입력

- (문장) "문화관광축제가 열렸다"

토큰나이저 후:

$$(t_1 t_2, \dots, t_n)$$

- [t1 = "문화", t2="관광", t3="축제", t4="가", t5="열렸다"]
- t_i 는 '문자'가 아닌 의미 단위(토큰)

각 토큰은 다음 임베딩의 합으로 표현된다.

$$x_i = e_i^{token} + e_i^{position} + e_i^{segment}$$

결과적으로 각 토큰은 768 개의 실수로 이루어진 벡터로 표현된다.

$$x_i \in \mathbb{R}^{768}$$

- 모델은 문자나 단어를 직접 이해하지 못하기에 숫자 벡터로 바꿔야 한다.
- e_i^{token} : 토큰 자체의 의미
- $e_i^{position}$: 문장 내 위치 정보
- $e_i^{segment}$: 문장 구분 정보(BERT 에서 주로 사용)

출력

$$H = \{h_1, h_2, \dots, h_n\}, h_i \in \mathbb{R}^{768}$$

- 출력도 "문자아 벡터"가 아니라 "토큰별 벡터"
- $\mathbb{R}$: 실수 전체의 집합
- $\mathbb{R}^{768}$: 실수 768 개를 한 묶음으로 가진 벡터들의 집합

[Token Sequence]

문장이나 데이터(입력)를 모델이 처리할 수 있도록 잘게 나눈 ‘토큰(token)’들의 순서 있는 집합

[Pooling layer]

풀링 계층(Pooling Layer)은 여러 개의 입력 값을 하나의 대표 값으로 요약하는 연산으로, 가변 길이의 데이터를 고정 차원의 표현으로 변환하는데 사용된다. 자연어 처리에서 문장은 단어의 개수와 관계없이 서로 비교 가능해야 하므로, 단어 단위로 생성된 표현을 문장 단위의 단일 벡터로 통합하는 과정이 필요하다. 이 역할을 수행하는 것이 바로 풀링 계층이다.

BERT 인코더는 입력 문장을 구성하는 각 토큰에 대해 문맥이 반영된 임베딩 벡터를 출력하지만, 그 결과는 문장 하나당 하나의 벡터가 아닌 토큰 개수만큼의 벡터 집합이다. 따라서 문장 간 의미적 유사도를 계산하거나 문서와 키워드를 비교하기 위해서는 이 토큰 임베딩들을 하나의 문장 임베딩으로 요약하는 추가적인 연산이 필요하다.

사용한 Ko-SBERT는 BERT 인코더의 출력 위에 풀링 계층을 추가하여 토큰 임베딩 집합을 문장 단위의 고정 차원 벡터로 변환한다. 특히 Mean Pooling 방식은 모든 토큰 임베딩의 평균을 문장 표현으로 사용함으로써, 특정 토큰에 과도하게 의존하지 않고 문장 전체의 의미 분포를 안정적으로 반영할 수 있다는 장점이 있다. 이 과정을 통해 문장은 고차원 의미 공간상의 하나의 벡터로 표현되며, 이후 문장 간 의미적 유사도는 벡터 간 거리 계산을 통해 효율적으로 수행될 수 있다.

[Siamese Network 구조(시암 네트워크)]

시암 네트워크는 동일한 파라미터를 공유하는 두 개의 인코더를 사용하여 입력 데이터 쌍을 각각 임베딩 공간으로 매핑하고, 이들 간의 거리 또는 유사도를 기반으로 학습을 수행하는 신경망 구조이다. 이 구조에서 두 인코더는 형태와 가중치가 완전히 동일하며, 입력만 서로 다를 뿐 동일한 변환 함수를 적용받는다. 이를 통해 서로 다른 입력에 대해서도 일관된 기준의 표현을 생성할 수 있다.

Ko-SBERT는 이러한 시암 네트워크 구조를 기반으로 문장 임베딩 학습을 수행한다. 각 문장은 동일한 BERT 인코더와 풀링 계층을 거쳐 고정 차원의 벡터로 변환되며, 문장 쌍 간의 의미적 유사도는 생성된 임베딩 벡터 간의 코사인 유사도를 통해 계산된다. 학습 과정에서는 의미적으로 유사한 문장 쌍은 임베딩 공간에서 서로 가까워지도록, 의미적으로 무관한 문장 쌍은 멀어지도록 모델의 파라미터가 조정된다.

이와 같은 시암 네트워크 구조의 장점은 문장을 독립적으로 임베딩할 수 있다는 점에 있다. 즉, 문장은 한 번만 벡터로 변환한 후 저장할 수 있으며, 이후의 유사도 계산은 벡터 간 연산만 수행되므로 대규모 문서 검색이나 키워드 추출과 같은 응용에서 높은 계산 효율을

제공한다. 이러한 특성으로 인해 Ko-SBERT는 문장 간 의미 비교가 핵심인 다양한 자연어 처리 과제에 효과적으로 활용될 수 있다.

[BERT 의 768 차원]

[1] 왜 하필 768 차원인가

BERT-base 에서 사용되는 768 차원(hidden size)은 임의로 선택된 값이 아니라, 모델 표현력과 계산 효율 사이의 균형을 고려하여 설계된 구조적 하이퍼파라미터이다. BERT 는 Transformer 인코더를 기반으로 하며, 각 토큰의 표현은 self-attention 을 통해 여러 의미적 관계를 병렬적으로 학습한다. 이때 hidden size 는 각 토큰이 담을 수 있는 정보의 “그릇 크기”에 해당한다.

BERT-base 는 12 개의 Transformer 레이어와 12 개의 attention head 를 가지며, 각 attention head 는 64 차원의 하위 공간(subspace)을 담당한다. 이 구조로부터 전체 hidden size 는 $12 \times 64 = 768$ 로 자연스럽게 결정된다. 즉, 768 차원은 attention 구조와 정합적으로 설계된 결과이며, 모델 내부 연산의 일관성과 병렬성을 유지하기 위한 선택이다

- Transformer 레이어(=인코더 레이어) : 문장을 단계적으로 이해하는 깊이를 담당
- Attention head : 각 단계에서 어떤 관계를 볼지를 병렬로 분담

[2] 차원이 클수록 항상 좋은가?

차원이 증가할수록 모델이 표현할 수 있는 의미적 자유도는 증가한다. 이는 더 미세한 의미 차이를 분리할 수 있음을 의미하며, 복잡한 문맥이나 미묘한 의미 차이를 다루는 데 유리하다. 그러나 차원이 무조건 클수록 성능이 향상되는 것은 아니다.

고차원 표현은 다음과 같은 단점을 동반한다. 첫째, 계산 비용과 메모리 사용량이 선형적으로 증가한다. 둘째, 학습 데이터가 충분하지 않을 경우 과적합(overfitting)이 발생할 가능성이 커진다. 셋째, 고차원 공간에서는 거리 유사도 계산이 불안정해질 수 있으며, 이는 ‘차원의 저주’로 알려진 문제이다.

따라서 hidden size 는 모델의 표현뿐만 아니라 학습 안정성과 실제 활용 가능성을 함께 고려하여 결정되어야 한다.

[3] 의미 공간 관점에서 본 차원 수의 해석

768 차원 의미 공간은 하나의 “구(sphere)”라기보다는, 학습 데이터에 의해 밀도가 형성된 고차원 분포 공간으로 해석하는 것이 적합하다. 코사인 유사도를 사용할 경우, 각 문장 벡터는 주로 방향에 의해 의미가 결정되며, 벡터의 크기(norm)은 상대적으로 중요하지 않다. 이 공간에서 의미적으로 유사한 문장들은 서로 가까운 방향으로 정렬되며, 서로 다른 주제나 문맥을 가진 문장들은 서로 다른 하위 공간(subspace)에 위치하게 된다. 따라서 768 차원은 단순한 수치가 아니라, 의미 분리를 가능하게 하는 충분한 차원의 표현 공간으로 가능하다.

MMR 기반 키워드 다양성 확보 - KeyBERT

[1. KeyBERT의 기본 개념]

[1-1] 키워드 추출 문제의 재정의

전통적인 키워드 추출은 단어의 출현 빈도 또는 통계적 가중치(TF-IDF 등)에 기반하여 이루어져 왔다. 이러한 접근은 계산적으로 효율적이지만, 문맥을 고려하지 못하고 의미적으로 유사한 단어들을 중복 추출하는 한계를 가진다. 특히 정책 문서나 사회 이슈와 같이 추상적이고 복합적인 의미 구조를 갖는 텍스트에서는, 단순 빈도 기반 키워드 추출이 문서의 핵심 의제를 충분히 반영하지 못하는 문제가 발생한다.

KeyBERT는 이러한 한계를 극복하기 위해 키워드 추출 문제를 통계적 선택 문제가 아닌, 의미 공간상의 최적 후보 선택 문제로 재정의한다.

[1-2] KeyBERT : 의미 기반 키워드 선택

KeyBERT는 문서와 키워드를 동일한 임베딩 공간으로 사상하여, 의미적 유사도에 기반한 키워드 선택을 수행하는 프레임워크이다. 이때 핵심 전제는 다음과 같다.

- 문서의 핵심 키워드는 문서 전체 의미 벡터와 가장 유사한 방향을 가지는 표현이다.

이를 위해 KeyBERT는 사전 학습된 문장 임베딩 모델(Ko-SBERT)를 사용하여, 문서 전체와 후보 키워드들을 모두 고차원 의미 공간 $\mathbb{R}^{768}$ 으로 매핑한다.

[1-3] KeyBERT의 처리 절차 개요

KeyBERT의 키워드 추출 과정은 다음과 같은 단계로 구성된다.

1. 후보 키워드 선택 : 형태소 분석기(Kiwi 등)를 활용하여 명사 또는 명사구 후보 집합 C 를 생성한다.
 2. 임베딩 생성 : 문서 전체는 하나의 문장 벡터 v_{doc} 로, 각 후보 키워드는 개별 벡터 v_c 로 Ko-SBERT 임베딩 공간에 매핑된다.
 3. 의미 유사도 계산: 문서 벡터와 후보 키워드 벡터 간의 코사인 유사도를 계산하여, 의미적으로 가장 정렬된 후보들을 선택한다.

이 과정에서 KeyBERT는 문자열 유사도나 빈도 정보가 아닌, 의미적 방향성을 기준으로 키워드를 평가한다.

[2. KeyBERT 와 MMR 을 이용한 키워드 최적화]

[2-1] KeyBERT 의 기본 원리: 임베딩 공간 내의 정렬(Alignment)

KeyBERT 는 문서 전체의 의미를 담은 벡터(v_{doc})와 개별 후보 키워드들의 벡터(v_{cand})를 동일한 Ko-SBERT 임베딩 공간($\mathbb{R}^{768}$)에 매핑한다.

키워드 추출:

- 문서의 핵심 키워드는 고차원 의미 공간상에서 문서 벡터와 가장 유사한 방향 벡터를 가진다.

이를 수학적으로 정식화하면, 후보 키워드 집합 C 중 유사도 상위 k 개를 선택하는 과정이다.

$$Top - k = \arg \max_{c \in C} (\cos(v_{doc}, v_c))$$

[2-2] MMR(Maximal Marginal Relevance)의 도입 배경

단순 유사도(cos)만으로 키워드를 추출할 경우, Ko-SBERT 공간의 비등방성(Anisotropy) 및 의미적 밀집성으로 인해 다음과 같은 한계가 발생한다.

- 의미적 중복: ‘지방자치’, ‘지방정부’, ‘지자체’와 같이 의미가 거의 동일한 단어들이 상위권을 독점함
- 정보의 편향성: 문서의 다양한 측면(예: 정책의 경제적 효과, 사회적 영향 등) 중 가장 지배적인 하나의 측면만 추출됨

이러한 의미 과밀 현상을 해소하기 위해 MMR(최대 한계 유사도) 알고리즘을 적용한다.

[2-3] MMR 의 수학적 정식화 및 파라미터 제어

MMR 은 다음의 목적 함수를 최대화하는 후보 D_i 를 반복적으로 선택하는 탐욕적 알고리즘이다.

$$MMR(D, K, S) = \arg \max_{D_i \in K \setminus S} [\lambda \cdot \text{sim}(D_i, D) - (1 - \lambda) \cdot \max_{D_j \in S} \text{sim}(D_i, D_j)]$$

- Relevance(적합도): 후보 D_i 와 문서 D 간의 코사인 유사도, 문서의 핵심 주제를 얼마나 잘 반영하는 지 측정한다.
- Diversity(다양성): 후보 D_i 와 이미 선택된 키워드 집합 S 내의 가장 유사한 키워드 D_j 간의 유사도. 이 값이 클수록 중복이 심함을 의미하므로 페널티를 부여한다.
- λ (Diversity Parameter)
 - $\lambda = 1$: 오직 문서와의 유사도만 고려(중복 허용)
 - $\lambda = 0$: 오직 기존 키워드와의 차별성만 고려(무작위성 증가)
 - $\lambda = 0.5 \sim 0.7$ (Optimal): 적합도와 다양성의 균형을 맞추는 스윗 스팟

[3-4] 기하학적 해석: 척력(Repulsive Force)의 작용

MMR 기반 키워드 추출은 기하학적으로 ‘중복 방지 척력’이 도입된 검색으로 해석할 수 있다.

- 첫 번째 키워드는 문서 벡터와 가장 가까운 점이 선택된다.
- 두 번째 키워드부터는 문서 벡터와 가깝더라도, 이미 선택된 점 주변의 ‘영향력 권역’에 들어온 후보들은 선택 확률이 낮아진다.
- 결과적으로 선택된 점들은 초구(S^{767}) 상에서 특정 지점에 물리지 않고, 문서 벡터 주위의 다양한 방향으로 골고루 분산된다.

[3-5] 기대 적용 효과

정책 의제의 다각적 포착

- 단순 추출 시 ‘코로나’ 관련 단어만 5 개가 뽑힐 상황에서, MMR 을 통해 ‘코로나(방역)’, ‘소상공인(경제)’, ‘백신(보건)’, ‘비대면(사회)’ 등 하나의 주제에 포함된 다양한 하위 의제(Sub-agendas)를 규현 있게 추출한다.

복합 명사 파편화 완화 및 의미적 응집

- Ko-SBERT 가 문맥적 유사도를 잡고 MMR 이 거리 조절을 수행함으로써, 형태소 분석 단계에서 발생할 수 있는 유사 단어들의 나열을 막고, 정보 밀도가 가장 높은 핵심 키워드들로만 구성된 집합을 도출한다.

[정보 추출 최적화 및 파라미터 튜닝(Optimization & Specifications)]

본 탐구에서는 국회 회의록의 정보를 손실 없이 추출하기 위해, 언어학적 제약 조건과 수학적 최적화를 결합하여 설계하였다.

[1] 언어학적 특징 추출(Linguistic Feature Extraction)

한국어 법률/행정 문서는 형태론적 복잡성이 높으므로, 단순 토큰화가 아닌 형태·의미론적 필터링(Morpho-Semantic Filtering)을 수행한다.

- Kiwi 기반의 형태소 제약: 정보 검색(Information Retrieval) 분야에서 실질적 의미를 지닌 ‘체언’ 위주의 추출을 위해 일반명사(NNG)와 고유명사(NNP)를 주된 분석 대상으로 설정하였다. 이는 문장의 기능어(조사, 어미 등)가 코사인 유사도 계산 시 노이즈로 작용하는 것을 방지하기 위함이다.

- 고정 불용어(Static Stop-words) 제거: 정책 문서의 특성상 빈번하게 등장하지만 지시 대상이 불분명한 단어(의원명, 관용적 행정 표현 등)를 도메인 지식 기반의 불용어로 정의하여 의미론적 드리프트(Semantic Drift, 의도치 않은 방향으로 임베딩이 쏠리는 현상)를 차단하였다.

[2] 하이퍼파라미터 그리드 서치(Hyper-parameter Grid Search)

모델의 변별력을 확보하기 위해 다음과 같은 독립 변수를 설정하여 성능 최적화를 도출하였다.

[2-1] n-gram Range: 정보 단위의 확장

- (1, 1) Unigram: 단일 개념(Concept) 추출에 용이하나, '문화산업'과 '진흥'이 분리되는 등 정보의 파편화가 발생할 수 있다.
- (1, 2) Bigram: '문화상품 제작', '지방자치단체 지원'과 같이 복합 명사구를 하나의 벡터로 인식한다. 이는 단순 단어보다 구체적인 정책적 맥락을 보존하는 데 유리하다.

[2-2] Diversity Parameter(λ): 다양성 임계치 조절

MMR 알고리즘 내의 λ 값은 적합도(Relevance)와 다양성(Diversity) 간의 트레이드오프(Trade-off)를 조절하는 가중치다.

- Low Diversity($\lambda \leq 0.4$): 문서 전체의 '중심 벡터'에 극도로 인접한 키워드만을 추출한다. 핵심 주제에 집중할 수 있으나 유의어 중복 문제가 발생한다.
- High Diversity($\lambda \geq 0.7$): 기하학적 거리가 먼 키워드를 우선 선택함으로써, 문서 내의 마이너 의제(Minor Agendas)를 포착하는 능력이 향상된다.

[2-3] 기술 용어 정의

용어	정의 및 기하학적 의미	프로젝트에서의 역할
Siamese Network	동일한 가중치를 공유하는 두 인코더 구조	문장 쌍 간의 상대적 거리를 학습하여 의미적 상동성 확보
Mean Pooling	모든 토큰 임베딩의 산술 평균을 구하는 연산	가변 길이의 문장을 고정된 768 차원 공간의 좌표로 응축
Anisotropy(비등방성)	벡터가 특정 방향으로 편향되어 분포하는 성질	단순 유사도 기반 추출 시 발생하는 키워드 중복의 근본 원인
MMR	한계 편익 극대화 원리를 적용한 재랭킹 알고리즘	고차원 초구상에서 선택된 점들 사이에 척력을 부여하는 중복 제거
Grid Search	가능한 파라미터 조합을 조사하는 최적화 기법	한국어 정책 문서에 가장 적합한 n-gram 과 diversity 의 균형점 탐색
Stop-word Filtering	정보 가치가 낮은 단어를 사전에 배제하는 전처리	임베딩 연산 시 코사인 유사도의 유효 변별력을 높임

[2-4] 추론 최적화

대량의 텍스트를 효율적으로 처리하기 위해 다음과 같은 전산적 최적화를 적용하였다.

- Vectorized Computation: CountVectorizer 를 Ko-SBERT 임베딩과 연동하여 후보 키워드 생성과 벡터화를 배치(Batch) 단위로 처리한다.
- Memory Efficiency: torch.no_grad() 컨텍스트를 사용하여 역전파(Backpropagation)에 필요한 연산 그래프 저장을 방지, 메모리 점유율을 약 50% 이상 절감하였다.
- Device Mapping: GPU(CUDA) 가속을 통해 고차원 행렬 연산(Matrix Multiplicaiton) 속도를 CPU 대비 비약적으로 향상시켰다.

[3] 최적 파라미터 설정의 근거

탐구 결과, $ngram_range = (1, 2)$ 와 $diversity=0.7$ 의 조합이 가장 우수한 정보 밀도를 보였다.

- 언어적 근거: 한국어 회의 용어는 복합 명사의 결합으로 이루어지는 경우가 많아(예: ‘문화예술법안심사소위원회’), 이를 개별 단어로 쪼갤 경우 고차원 공간에서 의미적 모호성이 증가한다. (1, 2) n-gram은 이러한 결합 명사의 응집력을 유지한다.
- 기하학적 근거: 회의록 문서는 하나의 문서 내에서 ‘행정 절차’, ‘예산 지원’, ‘기대 효과’ 등 다층적인 의제를 다룬다. $\lambda = 0.7$ 의 높은 다양성 파라미터를 적용함으로써, 중심 주제(centroid)에 몰려 있는 유사어 군집을 벗어나 초구 공간의 주변부에 위치한 세부 의제 벡터들을 효과적으로 샘플링할 수 있었다.

[코드]

```
import pandas as pd
from keybert import KeyBERT
from kiwipiepy import Kiwi
from sentence_transformers import SentenceTransformer
from sklearn.feature_extraction.text import CountVectorizer
import torch
import warnings

# Scikit-learn 의 불필요한 경고 메시지 무시 설정 (선택 사항)
warnings.filterwarnings("ignore", category=UserWarning, module="sklearn")

# [1] GPU 환경 체크 및 모델 로드
device = "cuda" if torch.cuda.is_available() else "cpu" # cuda 가 gpu 사용하는거 까지만 앞
print(f"Using device: {device}")

# 사용자 환경에 맞춘 모델 경로
model_path = r"C:\Users\Wngms0\mini-project\Wml_train\Wko-sbert"
model = SentenceTransformer(model_path, device=device)
kw_model = KeyBERT(model=model)
kiwi = Kiwi()

# [2] 테스트 문서 샘플
test_docs = [
    "문화예술법안심사소위원회의 소위원장 임오경입니다. 우리 소위원회는 12월 18일 문화체육관광부제 1차관 소관 6건 및 국가유산청 소관 2건의 법률안을 심사한 결과 2건을 원안 의결, 2건을 수정 의결하고 2건의 대안을 제안하기로 의결하였습니다. 의결 법률안에 대한 심사 결과를 보고드리겠습니다. 먼저 국어기본법 일부개정법률안(대안)은 임오경·조계원 의원이 각각 대표발의한 2건의 법률안을 통합 조정한 것으로 현재 운영 중이나 법적 근거가 미비한 국어문화원연합회와 세종 한국어 평가의 법적 근거를 마련하는 내용입니다. 다음, 김승수 의원이 대표발의한 문화산업진흥 기본법 일부개정법률안은 한국콘텐츠진흥원의 문화상품 제작 지원 등을 위한 대출계정을 설치하여 장기간 지속된 용자사업의법적 근거를 마련하는 것으로 원안대로 의결하였습니다. 다음, 정연욱 의원이 대표발의한 이스포츠(전자스포츠) 진흥에 관한 법률 일부개정법률안은 지방자치단체의 지원 대상에 이스포츠팀의 창단 및 운영과 학교 및 청소년 대상 이스포츠 활동과 진로교육 프로그램의 운영을 추가하려는 것으로 원안대로 의결하였습니다.",

    "전문위원입니다. 문화체육관광부 2차관 소관 총 6건의 법률안에 대한 검토의견을 간략히 요약해서 보고드리겠습니다. 1쪽 하단입니다. 의사일정 제 28항 민형배 의원이 대표발의한 국민체육진흥법 일부개정법률안은 장애인올림픽대회라는 명칭을 국제적으로 통용되는 패럴림픽대회로 정비하고 국민체육진흥기금의 사용처와 서울올림픽기념국민체육진흥공단의 사업 범위에 서울패럴림픽 기념사업을명시하는 내용입니다. 장애인올림픽대회는 국제패럴림픽위원회가 주최하는 대회로 1989년 위원회가 설립된이후 패럴림픽이라는 용어로 사용되고 있다는 점에서 용어정비는 타당성이 있다고 보았습니다. 다만 서울올림픽기념국민체육진흥공단이 서울패럴림픽대회 기념사업도 담당하게 될경우 올림픽 기념만을 명시하고 있는 공단의 공식 명칭 및 목적도 변경될 필요가 있다고보았습니다. 3쪽입니다. 다음, 의사일정 제 30항 임오경 의원이 대표발의한 문화관광축제 육성 및 지원에 관한법률안은 문화관광축제를 국가 관광정책력 제고와 지역발전의 전략적 플랫폼으로 육성하기 위하여 문화관광축제와 글로벌 축제의 지정 근거를 마련하고 문화관광축제 기본계획및 시행계획의 수립, 실태조사, 전문인력 양성, 전담 조직의 설치, 후원 및 기부금품의접수 등 행정적·재정적 지원체계를 구축하고자 하는 법안입니다. 동 법안은 축제 정책의 공공성·지속가능성을 강화하고 문화관광축제의 발전에 기여할것으로 기대되나 지역축제의 정의와 범위, 문화관광축제 전담 조직의 기부금품 접수조항 등에 대해서는 추가적인 검토가 필요할 것으로 보았습니다. 이하 내용은 생략하도록 하겠습니다. 감사합니다.",

    "79 페이지를 보게 되면 수정의견의 24 조의 2 1 항에 대한 의견입니다. 79 페이지 당구장 표시에 '의무·재량여부 논의 필요'라고 돼 있는데요. 보통의 경우에 그 법령에서의무규정으로두는경우에는의무규정이후에의무를이행하지않을때벌칙규정을 두게 됩니다. 그런데 이것은 지금 산업통상부장관에게 어떠한 의무를 부여하는 것이라서 만약에 이렇게 의무를 부여했음에도 불구하고 이 법의 취지에 맞게 충분한 의무 이행을 하지 않는다고 나중에 추후에 판단을 하게 되면 나중에 산자부장관이 책임을 질 가능성이 좀 있어요. 그 정도 고려를 해서 입법을 하시는 게 좋을 것 같습니다. 그렇지지는 않아요?",

    "검토의견과 수정의견에 대해서 동의합니다."
]

# [3] Kiwi 기반의 커스텀 토큰라이저 함수 (불용어 필터 포함)
def kiwi_tokenizer(text):
    # 키워드에서 제외할 단어들 (의원 이름, 일반 행정 용어 등)
    stop_words = ['임오경', '조계원', '김승수', '정연욱', '민형배', '전문위원', '의원', '경우', '사항', '이하', '내용']
```

```

tokens = kiwi.analyze(text) # 키워드로 추출
result = []
if tokens:
    for t in tokens[0][0]:
        # 일반명사(NNG), 고유명사(NNP), 외국어(SL) 중 2 글자 이상 추출
        if t.tag in ['NNG', 'NNP', 'SL'] and len(t.form) > 1:
            if t.form not in stop_words:
                result.append(t.form)
    return result

# [4] 실험 파라미터
ngram_ranges = [(1,1), (1,2)]
diversities = [0.2, 0.4, 0.7]
top_n = 5

# [5] 결과 저장 리스트
results = []

print("개선된 키워드 추출 테스트 시작 (Warning 제거 버전)...")

# [6] 키워드 추출 실행
with torch.no_grad(): # 자동 미분 엔진을 일시적으로 비활성화하여 기울기 추적을 중단시키는 컨텍스트 관리자,
Autograd 엔진 자체를 끄는 것으로, 연산 기록을 중단시켜 메모리와 속도 효율을 높인다.

    for doc_idx, doc in enumerate(test_docs):

        for nr in ngram_ranges:

            # token_pattern=None 설정을 통해 UserWarning 방지
            vectorizer = CountVectorizer(
                # 문서에서 단어의 빈도수를 계산해서 문서 단어 행렬을 만들어주는 작업을 하는 모듈
                tokenizer=kiwi_tokenizer,
                ngram_range=nr,
                token_pattern=None
            )

            for div in diversities:
                try:
                    # KeyBERT 핵심 로직
                    keywords = kw_model.extract_keywords(
                        docs=doc,
                        vectorizer=vectorizer,
                        use_mmr=True,
                        diversity=div,
                        top_n=top_n
                    )
                except Exception:
                    # 문서가 너무 짧아 추출이 불가능할 경우 빈 리스트 처리
                    keywords = []

            # 결과 포매팅 및 저장
            res_tags = [f"{k}({s:.2f})" for k, s in keywords]
            results.append({
                "doc_id": doc_idx + 1,
                "ngram_range": nr,
                "diversity": div,
                "keywords": ", ".join(res_tags)
            })

# [7] 결과 출력
df = pd.DataFrame(results)
pd.set_option('display.max_colwidth', None)

for i in range(len(test_docs)):
    print(f"Wn--- [문서 {i+1}]번 실험 결과 ---")

```

```
# 가독성을 위해 관련 파라미터와 키워드만 출력
sub_df = df[df['doc_id'] == i+1][['ngram_range', 'diversity', 'keywords']]
print(sub_df.to_string(index=False))
```

[코드 설명]

```
def kiwi_tokenizer(text):
    # 키워드에서 제외할 단어들 (의원 이름, 일반 행정 용어 등)
    stop_words = ['임오경', '조계원', '김승수', '정연욱', '민형배', '전문위원',
                  '의원', '경우', '사항', '이하', '내용']

    tokens = kiwi.analyze(text) # 키워로 추출
    result = []
    if tokens:
        for t in tokens[0][0]:
            # 일반명사(NNG), 고유명사(NNP), 외국어(SL) 중 2 글자 이상 추출
            if t.tag in ['NNG', 'NNP', 'SL'] and len(t.form) > 1:
                if t.form not in stop_words:
                    result.append(t.form)
    return result
```

kiwi.analyze(text)

- 문장을 형태소 + 품사로 분석한 “후보 결과 목록”을 변환하는 함수
- tokens[0] : 가장 점수가 높은 분석 결과
- tokens[0][0] : 그 분석 결과의 형태소 리스트

[참고 자료]

Kiwi : <https://github.com/bab2min/Kiwi>